

Worksheet — 1.3 Exchanging Data — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

1. **Key-term match** A-4, B-3, C-6, D-1, E-2, F-5, G-8, H-7

2. **RLE** - (a) (R,4) (G,2) (B,3) (W,5) — i.e. 4 R, 2 G, 3 B, 5 W. - (b) A A A B A A A A C C (3 A, 1 B, 4 A, 2 C). - (c) Data with **few/no runs** of repeats — e.g. a detailed photograph or already-compressed text. Each unique value still needs a count, so the file *grows* (no repetition to exploit). RLE is lossless and only shrinks files with long runs.

3. **Symmetric vs asymmetric** - (a) **same** key; **key distribution / key exchange** problem. - (b) **public** key; **private** key. - (c) **private** key. - (d) **slower**. - (e) **symmetric** session key; then **symmetric** encryption for bulk data.

4. **Normalisation** - (a) The **Courses** column holds multiple values ("Maths, Physics") — a **repeating group** / non-atomic value. To reach 1NF, make values atomic: one course per row (or move courses to a separate table) and ensure a suitable primary key. - (b) **Transitive dependency** (TutorName depends on TutorID, a non-key attribute, not directly on the key). This breaks **3NF**. (Partial dependencies break 2NF; transitive break 3NF.) - (c) Sample 3NF design (primary keys in **bold**, foreign keys marked FK): - **Student**(**StudentID** , Name, TutorID (FK)) - **Tutor**(**TutorID** , TutorName) - **Course**(**CourseID** , CourseName) - **StudentCourse**(**StudentID (FK)**, **CourseID (FK)**) — linking/junction table with a **composite** primary key to resolve the many-to-many between students and courses.

5. **SQL** (accept reasonable variations / casing)

```
-- (a)
SELECT Name, YearGroup
FROM Student
WHERE YearGroup = 12
ORDER BY Name ASC;

-- (b)
SELECT Student.Name, Tutor.TutorName
FROM Student
JOIN Tutor ON Student.TutorID = Tutor.TutorID;

-- (c)
UPDATE Student
SET YearGroup = 13
WHERE StudentID = 'S01';
```

6. **TCP/IP layer matching** - Application: **HTTP, SMTP, FTP** - Transport: **TCP, UDP** - Network / Internet: **IP** - Link / Data link: **Ethernet, Wi-Fi**

7. Short answer - (a) Any two of: **source IP address**, **destination IP address**, a **sequence/packet number** (for reassembly in the right order). (Also acceptable: error-check/checksum, TTL.) - (b) **DNS translates a domain name (e.g. example.com) into an IP address** so the device can locate the server. Caching stores recently looked-up results, so repeat lookups are **faster** and reduce traffic/load on DNS servers. - (c) PageRank treats each incoming link as a "vote" of importance; a high-ranked page passes on **more of its rank** through its links, so its vote counts for more. (Damping factor ~ 0.85 ; iterates until values converge.) - (d) Client-side checks can be **viewed, bypassed or disabled** by the user (e.g. editing the page / disabling JavaScript), so they are not secure. Server-side checks are hidden and authoritative, so security-critical checks must run there.

Challenge box (model points) - Hashing is **one-way**: even if the database is stolen, attackers cannot reverse the hash back to the password (unlike encryption, which is reversible with a key). So the plaintext password is never stored. - It is **deterministic**: the same password always produces the same hash. On login, the entered password is hashed and **compared to the stored hash** — if they match, access is granted; the password itself is never compared. - A good hash function has **few collisions** (different inputs rarely produce the same hash), so a wrong password is very unlikely to produce a matching hash. (Real systems also add a *salt*.)